The digithings book

Printed from the same files the web pages are built from.

contents

01 Why the book exists · 2
The prose pages answer "how do I use this"; the book answers "why is it built this way", in the order that makes the answer readable.

02 How the book is built · 3
One chapter file becomes one page and one print page, from one diagram object. The pipeline, and the one rule that keeps the two the same.

03 Reading and printing · 6
Pan and zoom on screen, paginate on paper, and build the file in CI so the printed copy is a build artifact rather than somebody's browser.

01

Why the book exists

The prose pages answer "how do I use this"; the book answers "why is it built this way", in the order that makes the answer readable.

The site has a docs section, and the docs section is good at its job: it tells you the flags, the endpoints, and the order to type things in. What it does not do is explain the shape. Every question of the form *why is it like this* ends either unanswered or answered in a comment thread where the next reader cannot find it.

What a book is for here

A book is not documentation. Documentation is indexed by the thing you are trying to do; a book is indexed by the thing you are trying to understand. The same fact appears in both, but the book is allowed to spend a page on it.

Three things follow from that:

- **It is versioned as prose.** A chapter is a file with a header, not a row in a table that a migration can rewrite out from under you.

Skip to content

d■

- **It is printable.** A chapter is read on a screen and on paper, from one file, so the diagrams are the same objects in both.
- **It has an order that the author chose.** The `order` field in the chapter header decides the sequence. Renaming a file does not move a chapter.

What it is not

The book is not a second home for the API reference. That surface changes when a route changes, and it is generated. Prose that has to be regenerated is not prose.

Nor is the book a place for anything we cannot show. Every claim here is about code in the repository, and every diagram is rendered from the same source the page is written against.

How to read it

Start at the first chapter. The later ones assume the vocabulary the early ones introduce, which is the ordinary price of a book and the reason it is ordered rather than indexed.

If you are looking for a flag, you want the docs. If you are looking for a reason, stay here.

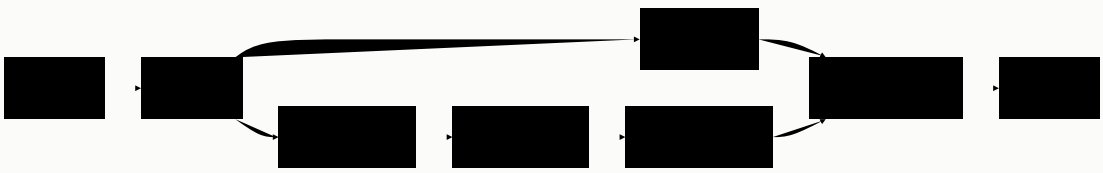
02

How the book is built

One chapter file becomes one page and one print page, from one diagram object. The pipeline, and the one rule that keeps the two the same.

Every chapter is one `.mdx` file under `content/book`. It opens with a header that says three things — the title, the order, and a one-paragraph summary — and the rest is ordinary prose with ordinary markdown.

The pipeline



select the figure to pan and zoom

Skip to content

four steps and one of them is the only place a diagram is ever rendered.

d■

The one-address rule

A diagram's name is a hash of its source. That hash decides two paths, and both are produced by the same function:

- the **URL** the page renders — `/book/diagrams/<id>.svg`
- the **file** the build writes — `public/book/diagrams/<id>.svg`

Because there is a single address, the page you read on screen and the page that comes out of the printer load the same object. There is no second render and no opportunity for the two to drift.

Editing a diagram changes its source, which changes the hash, which changes the address. A stale diagram cannot survive under an old name.

Why not render in the browser

A browser-side renderer would be less code and would give a sharper image at every zoom level. It would also give two different things: the screen would show a live render and the print would show whatever the print pipeline could recreate. For a book, the printed copy is the one that is still readable in ten years, and it should not depend on a client bundle staying alive.

Why a plain image

The rewritten chapter contains an `<img>`, not a component. An image needs no client code, prints identically under any paginating engine, and is readable by a screen reader through its alt text. The caption is the alt text, so the two say the same thing.

Headings and anchors

Each heading gets one id: a slug of its own words. There is no author-chosen form, because MDX reads `{` as the start of an expression and a heading carrying `{#custom}` fails to compile — which was measured, not assumed.

Repeated headings get a `-2` suffix in document order, so two sections called `Notes` produce two working entries rather than one link that jumps to the wrong place.

The table of contents holds only levels two to four. A level-one heading is the chapter title, which the page already shows, and anything deeper is a clause that would turn the rail into a wall of links.

03

Reading and printing

Skip to content

The book is read two ways and both are supported deliberately.

On screen

A chapter page carries the table of contents for that chapter as a rail, and the full book list as the previous/next pair at the foot of the page. Diagrams are ordinary images until you ask for them to be bigger.

Dragging a diagram opens it: the image pans and zooms under the pointer, with the wheel and with a pinch. It is an interaction on top of an image, so the image is still there when the interaction is not — no script, no figure.

On paper

Printing a chapter uses a stylesheet that the print engine honours rather than fights:

- running headers carry the chapter title and the book title;
- page numbers sit in the outer margin, which alternates by page parity;
- the table of contents on the printed front matter carries page references, resolved after the document is paginated.

The last point is the one worth explaining. A page number is not knowable until the engine has decided where the pages break, so it is not written into the source. The engine fills it in, after pagination, by looking up the anchor each entry points at. A number written by hand would be wrong the moment a paragraph grew a line.

Why the print step is a build step

The printed book is produced by a headless browser in CI, and the resulting file is uploaded as an artifact. It is not produced by a person pressing print.

That decision costs a browser in the pipeline and buys three things: the file is reproducible from a commit, the same input always yields the same output, and nobody's machine settings — paper size, margins, "fit to page", a stray header and footer — end up in the printed book.

What the two outputs share

Everything that matters. The diagrams, the anchors, the running headers, the page

Skip to content

. A change to the page that improves reading improves the print, because

Contact | Privacy | X | LinkedIn

[Live]
digichat
digiquant
dashboard

[Product]
Docs
API
Wiki
Services

[Company]
About
Team
Security
Changelog

[Connect]
GitHub
contact@digithings.ai

[Legal]
Privacy

© 2026 digithings · open core

paginating...

The digithings book

Skip to content

ed from the same files the web pages are built from.

contents

01 Why the book exists · 2
The prose pages answer "how do I use this"; the book answers "why is it built this way", in the order that makes the answer readable.

02 How the book is built · 3
One chapter file becomes one page and one print page, from one diagram object. The pipeline, and the one rule that keeps the two the same.

03 Reading and printing · 6
Pan and zoom on screen, paginate on paper, and build the file in CI so the printed copy is a build artifact rather than somebody's browser.

01

Why the book exists

The prose pages answer "how do I use this"; the book answers "why is it built this way", in the order that makes the answer readable.

The site has a docs section, and the docs section is good at its job: it tells you the flags, the endpoints, and the order to type things in. What it does not do is explain the shape. Every question of the form *why is it like this* ends either unanswered or answered in a comment thread where the next reader cannot find it.

What a book is for here

Skip to content

d■

A book is not documentation. Documentation is indexed by the thing you are trying to do; a book is indexed by the thing you are trying to understand. The same fact appears in both, but the book is allowed to spend a page on it.

Three things follow from that:

- **It is versioned as prose.** A chapter is a file with a header, not a row in a table that a migration can rewrite out from under you.
- **It is printable.** A chapter is read on a screen and on paper, from one file, so the diagrams are the same objects in both.
- **It has an order that the author chose.** The `order` field in the chapter header decides the sequence. Renaming a file does not move a chapter.

What it is not

not prose.

Nor is the book a place for anything we cannot show. Every claim here is about code in the repository, and every diagram is rendered from the same source the page is written against.

How to read it

Start at the first chapter. The later ones assume the vocabulary the early ones introduce, which is the ordinary price of a book and the reason it is ordered rather than indexed.

If you are looking for a flag, you want the docs. If you are looking for a reason, stay here.

02

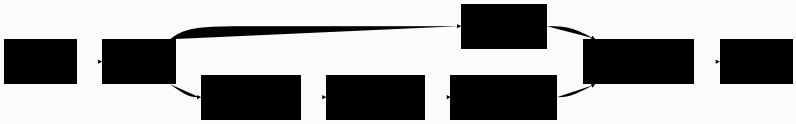
How the book is built

Skip to content

hapter file becomes one page and one print page, from one diagram object. The pipeline, and the one rule that keeps the two the same.

Every chapter is one `.mdx` file under `content/book`. It opens with a header that says three things — the title, the order, and a one-paragraph summary — and the rest is ordinary prose with ordinary markdown.

The pipeline



select the figure to pan and zoom

There are four steps and one of them is the only place a diagram is ever rendered.

The one-address rule

A diagram's name is a hash of its source. That hash decides two paths, and both are produced by the same function:

- the **URL** the page renders — `/book/diagrams/<id>.svg`
- the **file** the build writes — `public/book/diagrams/<id>.svg`

Because there is a single address, the page you read on screen and the page that comes out of the printer load the same object. There is no second render and no opportunity for the two to drift.

Editing a diagram changes its source, which changes the hash, which changes the address. A stale diagram cannot survive under an old name.

Skip to content

Why not render in the browser

A browser-side renderer would be less code and would give a sharper image at every zoom level. It

would also give two different things: the screen would show a live render and the print would show whatever the print pipeline could recreate. For a book, the printed copy is the one that is still readable in ten years, and it should not depend on a client bundle staying alive.

Why a plain image

The rewritten chapter contains an `<img>`, not a component. An image needs no client code, prints identically under any paginating engine, and is readable by a screen reader through its alt text. The caption is the alt text, so the two say the same thing.

Headings and anchors

Each heading gets one id: a slug of its own words. There is no author-chosen form, because MDX reads `{` as the start of an expression and a heading carrying `{#custom}` fails to compile — which was measured, not assumed.

Repeated headings get a `-2` suffix in document order, so two sections called `Notes` produce two working entries rather than one link that jumps to the wrong place.

The table of contents holds only levels two to four. A level-one heading is the chapter title, which the page already shows, and anything deeper is a clause that would turn the rail into a wall of links.

Skip to content

03

Reading and printing

Pan and zoom on screen, paginate on paper, and build the file in CI so the printed copy is a build artifact rather than somebody's browser.

The book is read two ways and both are supported deliberately.

On screen

A chapter page carries the table of contents for that chapter as a rail, and the full book list as the previous/next pair at the foot of the page. Diagrams are ordinary images until you ask for them to be bigger.

Dragging a diagram opens it: the image pans and zooms under the pointer, with the wheel and with a pinch. It is an interaction on top of an image, so the image is still there when the interaction is not — no script, no figure.

On paper

Printing a chapter uses a stylesheet that the print engine honours rather than fights:

- running headers carry the chapter title and the book title;
- page numbers sit in the outer margin, which alternates by page parity;
- the table of contents on the printed front matter carries page references, resolved after the document is paginated.

The last point is the one worth explaining. A page number is not knowable until the engine has decided where the pages break, so it is not written into the source. The engine fills it in, after pagination, by looking up the anchor each

Skip to content

entry points at. A number written by hand would be wrong the moment a paragraph grew a line.

Why the print step is a build step

The printed book is produced by a headless browser in CI, and the resulting file is uploaded as an artifact. It is not produced by a person pressing print.

That decision costs a browser in the pipeline and buys three things: the file is reproducible from a commit, the same input always yields the same output, and nobody's machine settings — paper size, margins, "fit to page", a stray header and footer — end up in the printed book.

What the two outputs share

Everything that matters. The diagrams, the anchors, the running headers, the page geometry. A change to the page that improves reading improves the print, because there is only one page.